

Introdução a arquitetura de Microserviços

Engenharia de Qualidade e Confiabilidade

Prof. Me. Sergio Souza Novak — UniRV

21 de maio de 2026



- 1 O que são Microserviços?
- 2 Por que adotar Microserviços?

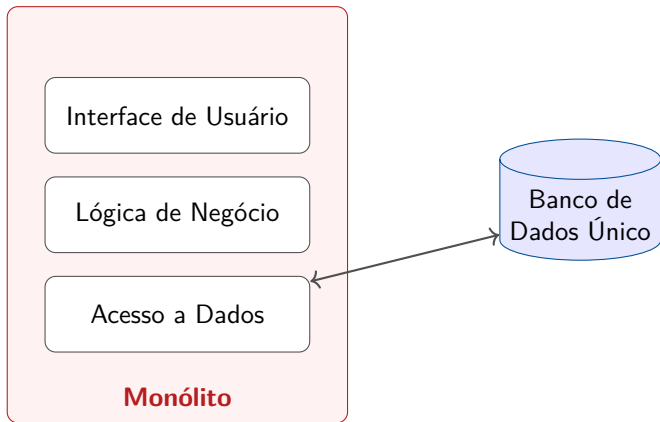
- 3 Vantagens dos Microserviços
- 4 Desafios dos Microserviços

O que são Microsserviços?

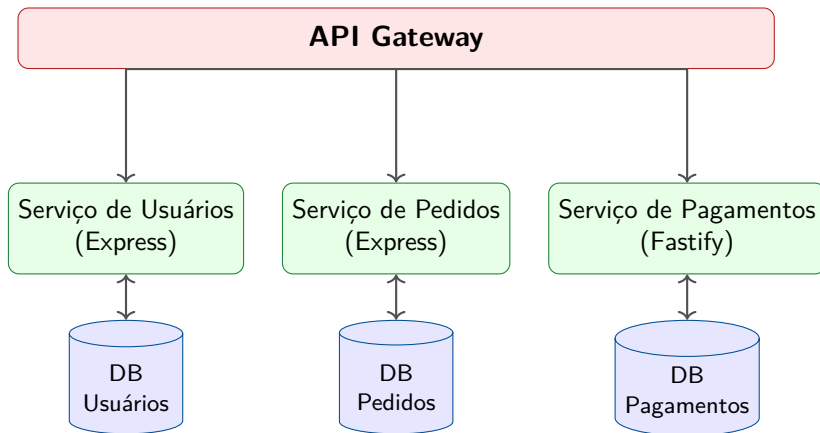
- Arquitetura onde uma aplicação é composta por serviços pequenos e independentes
- Cada serviço executa uma única funcionalidade de negócio
- Serviços se comunicam via APIs leves (ex.: HTTP/REST, mensageria)
- Cada microsserviço pode ser desenvolvido, implantado e escalado de forma independente



Arquitetura Monolítica



Sistema de Microserviços (Node.js)



Exemplo Inicial: Serviço de Usuários com Express

Um microserviço de usuários funcional extremamente simples em **Node.js / Express**.

```
const express = require('express');
const app = express();
app.use(express.json());

// Banco de dados em memoria para simplificar
const users = [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }];

app.get('/api/users', (req, res) => {
  res.json(users);
});

app.get('/api/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).send('Usuario nao encontrado.');
```

res.json(user);

```
});

app.listen(3001, () => console.log('Servico de Usuarios na porta 3001'));
```

Por que adotar Microserviços?

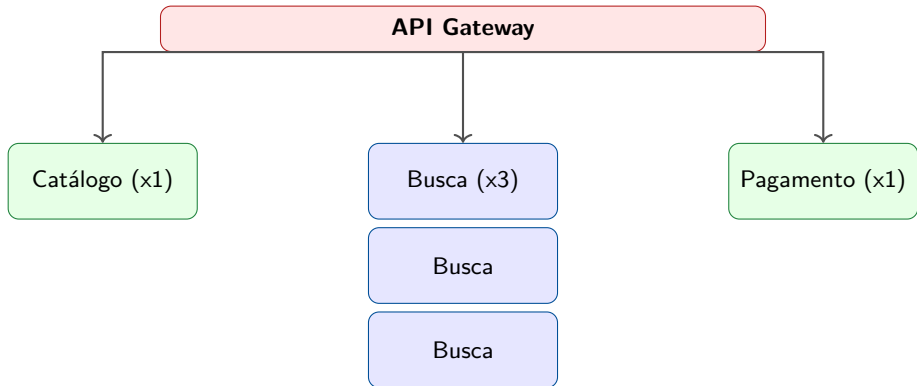
Gigantes da indústria (Netflix, Twitter, Amazon, eBay, Uber) migraram de monolíticos para microserviços devido a:

- **Dificuldade de escalabilidade** – escalar o sistema inteiro quando apenas uma parte precisa de mais recursos
- **Falta de eficiência** – recursos ociosos em partes não-demandadas
- **Velocidade lenta de desenvolvimento** – equipes pisam umas nas outras no mesmo código
- **Dificuldade em adotar novas tecnologias** – preso a uma stack única

Monólito → Microserviços: um passo natural no processo de escalar uma aplicação.

Vantagens: Escalabilidade

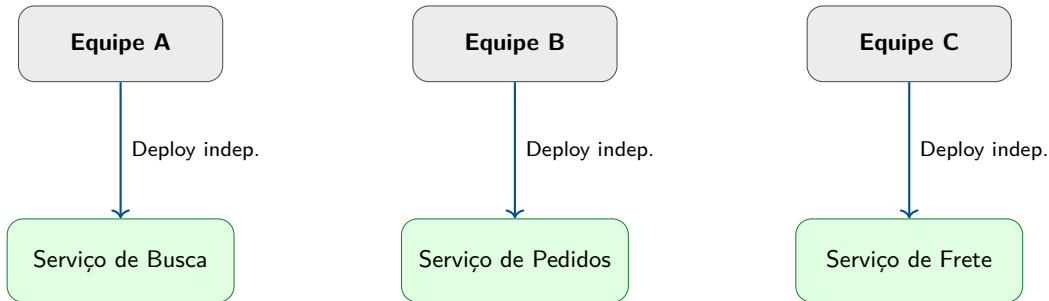
Escala horizontal e vertical de forma independente por serviço.



Vantagens: Produtividade

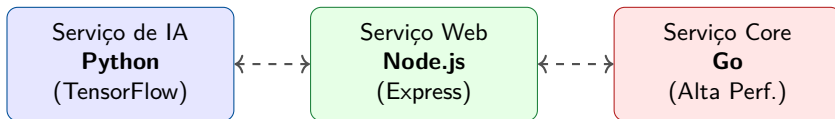
Autonomia das Equipes

Em arquiteturas de microsserviços, equipes pequenas e multidisciplinares podem desenvolver, testar e implantar serviços de forma independente.



Vantagens: Inovação Tecnológica

Cada serviço pode usar a tecnologia mais adequada; trocas graduais sem impacto.



Vantagens: Resiliência

Falha em um serviço não derruba toda a aplicação.



A migração para microsserviços traz desafios significativos:

- **Infraestrutura sofisticada** – orquestração (Kubernetes), service discovery, monitoramento, logging centralizado
- **Mudança organizacional** – estrutura de equipes precisa ser alterada para acompanhar a arquitetura
- **Risco de segregação** – equipes isoladas podem levar à dispersão de conhecimento
- **Padronização** – necessidade de definir contratos de API, padrões de comunicação e governança
- **Confiança e disponibilidade** – garantir que cada serviço seja confiável e esteja disponível

Monólito vs Microserviços

Característica	Monólito	Microserviços
Implantação	Única	Independente por serviço
Escalabilidade	Vertical (tudo)	Horizontal (por serviço)
Stack tecnológica	Única	Heterogênea
Acoplamento	Alto	Baixo
Tamanho da equipe	Grande	Pequenas equipes
Complexidade operacional	Baixa	Alta (orquestração)
Resiliência	Baixa (falha total)	Alta (falha isolada)

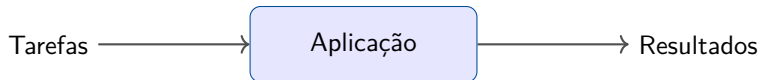
Atenção

Microserviços não são uma bala de prata. A decisão deve considerar o tamanho, a maturidade e as necessidades do negócio.

Dimensionando uma Aplicação

Objetivo de qualquer aplicação

Processar tarefas de alguma natureza de forma **eficiente**.

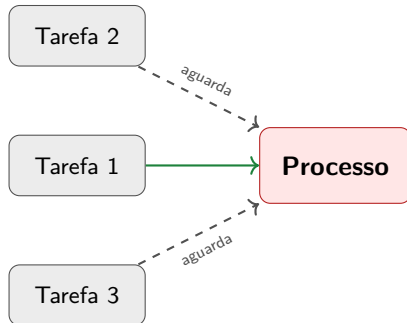


Como garantir que isso aconteça de forma eficiente?

O Problema do Processo Único

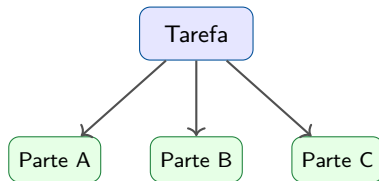
Processo sequencial

Um único processo escolhe uma tarefa por vez, conclui todas as etapas (ou falha!) e então passa para a próxima. Isso **não é eficiente!**



Introduzindo Simultaneidade

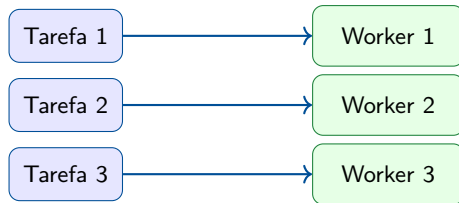
Para tornar a aplicação eficiente, introduzimos **simultaneidade**: cada tarefa pode ser dividida em partes menores.



Processamento concorrente

Dividir e Conquistar: Segmentação

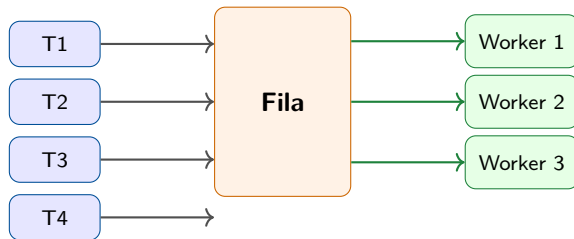
Além da simultaneidade, a **segmentação** permite que várias tarefas sejam processadas **em paralelo**.



Cada tarefa em um worker dedicado

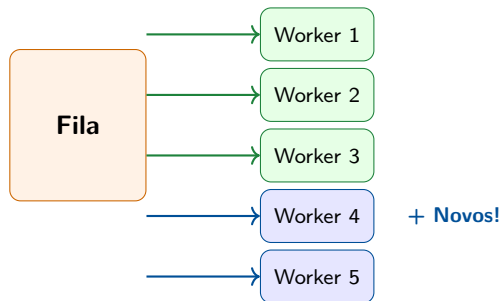
Pool de Workers

Se tivermos várias tarefas, podemos processá-las **todas ao mesmo tempo** enviando-as para um grupo de workers.



Escalando sob Demanda

Precisa processar **mais tarefas**? Basta acrescentar **workers adicionais** sem afetar a eficiência do sistema.



Limitação do Monólito

Simultaneidade e segmentação são **difíceis de suportar** em uma aplicação grande que precisa:

- Ser implantada em **todos os servidores**
- Processar **qualquer tipo de tarefa**

Escalar = Mais hardware

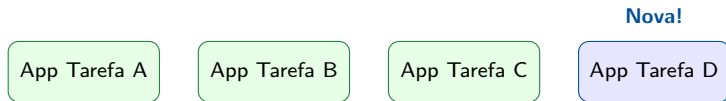
A única maneira de dimensioná-la é **aumentar o hardware** no qual a aplicação é implantada (escala vertical).



A Solução: Dividir a Aplicação

A melhor maneira de dimensionar é **dividir a aplicação em muitas aplicações independentes pequenas**, cada uma executando um tipo de tarefa.

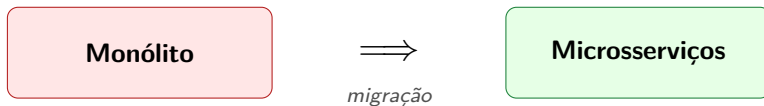
- Precisa acrescentar uma nova etapa? **Crie uma nova aplicação!**
- Precisa lidar com mais tráfego? **Acrescente mais workers!**



Empresas que migraram

Amazon, Twitter, Netflix, eBay e Uber executam aplicações em **milhares** (até centenas de milhares) de servidores.

Suas aplicações tornaram-se **monólitos** e passaram a enfrentar desafios de escalabilidade.



Superaram os desafios adotando microserviços.

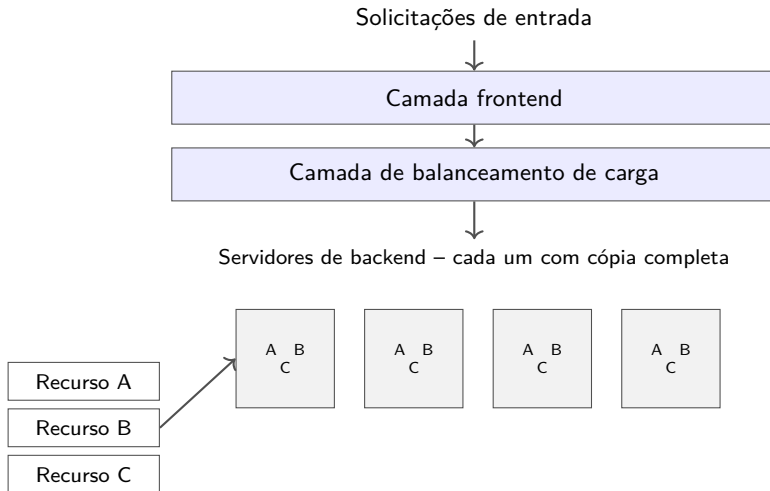
Definição

Um microserviço é uma **pequena aplicação** que executa uma **única tarefa** e o faz com eficiência.

Características:

- Pode ser facilmente **substituído**
- É desenvolvido e implantado de forma **independente**
- Não sobrevive sozinho — *nenhum microserviço é uma ilha*
- Faz parte de um **sistema maior**, funcionando junto a outros microserviços

Arquitetura Monolítica Detalhada

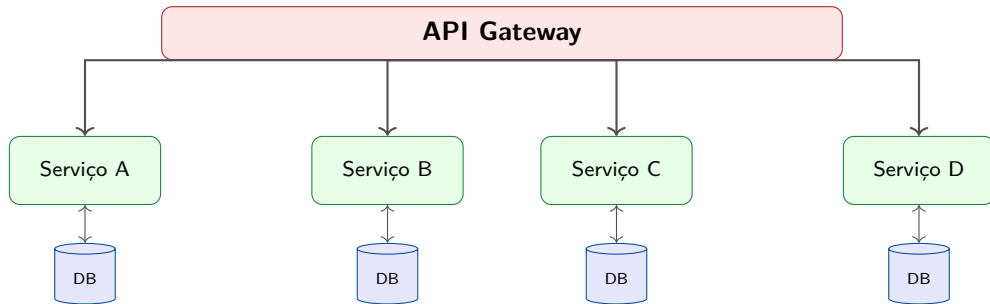


*Todos os recursos em
uma única aplicação*

Ecosistema de Microserviços

Objetivo

Construir um conjunto de pequenas aplicações, cada uma responsável por **uma função**, autônomas e independentes.



Cada serviço: autônomo, independente, autossuficiente

Vantagens de Adotar Microsserviços

- Menor **defasagem técnica**
- Melhor **produtividade** e velocidade do desenvolvedor
- Melhor **eficiência de teste**
- Maior **escalabilidade**
- Facilidade de **implantação**

Caminho natural

Empresas geralmente adotam microsserviços **depois** de construir uma aplicação monolítica e encontrar desafios organizacionais e de escalabilidade.

Monólito



Desafios



Microsserviços

Dificuldades em Dividir o Monólito

As dificuldades dependem totalmente da **complexidade da aplicação monolítica**.

- Aplicação com muitos recursos → muito esforço de arquitetura
- Necessidade de **reorganizar e reestruturar equipes**
- A decisão deve ser um **esforço que envolva toda a empresa**

Atenção

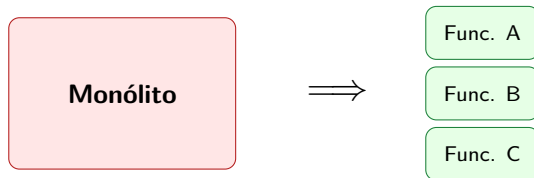
Complexidade adicional é introduzida pela necessidade de reestruturação organizacional.

Etapa 1: Identificar os Componentes

A etapa mais difícil

Localizar com precisão as principais funcionalidades do monólito e dividi-las em pequenos componentes independentes.

- Existem muitas maneiras **corretas** de dividir
- Existem **muito mais** maneiras **erradas** de fazê-lo
- Microsserviços devem ser **tão simples quanto possível**

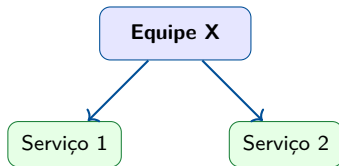


Etapa 2: Reestruturar as Equipes

Cada microserviço deve ter sua própria equipe de engenharia.

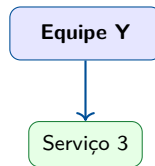
Método 1: Equipe por serviço

Uma equipe dedicada para cada microserviço. Tamanho determinado pela complexidade do serviço.



Método 2: Equipe por domínio

Vários serviços atribuídos a uma equipe organizada em torno de produtos ou domínios de negócio.



Risco do Método 2

Ao atribuir vários serviços a uma equipe, é preciso garantir que os desenvolvedores **não sejam sobrecarregados**.

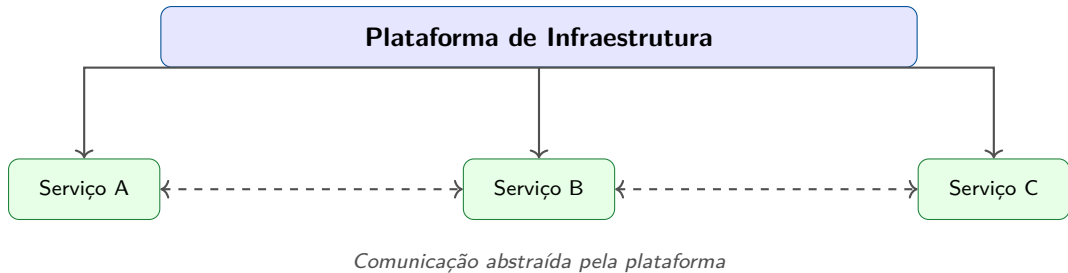
Riscos:

- **Fadiga operacional** – excesso de tarefas de manutenção
- **Fadiga de interrupções** – alertas e incidentes constantes
- **Queda de qualidade** – pressa para entregar pode gerar bugs

Equipes saudáveis = Microsserviços saudáveis

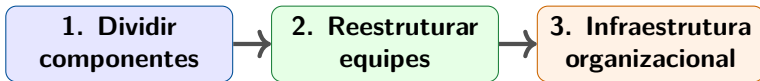
Etapa 3: Criar o Ecossistema de Infraestrutura

As equipes de infraestrutura devem fornecer uma **plataforma estável** que abstraia as complexidades das interações entre microsserviços.



As Três Etapas da Migração

- ❶ **Dividir** a aplicação em componentes de serviço
- ❷ **Reestruturar** as equipes de engenharia
- ❸ **Desenvolver** a infraestrutura organizacional



Somente após essas etapas a migração pode começar.

Estratégias de Migração

Estratégia 1: Extrair do monólito

Extrair o código relevante diretamente do monólito para um serviço separado e **replicar o tráfego** até validar.

Estratégia 2: Construir do zero

Construir o serviço a partir do zero e redirecionar o tráfego após **testes adequados**.

Essencial

Uma migração bem-sucedida exige execução e planejamento **minuciosos, cuidadosos e meticulosamente documentados**. Uma migração completa de um grande monólito pode levar **muitos anos**.

Começar Direto com Microsserviços?

Cuidado!

Pode parecer melhor começar já com microsserviços, mas:

- Empresas pequenas **não têm a infraestrutura** necessária
- Uma boa arquitetura requer infraestrutura **estável e complexa**
- Manter essa infraestrutura exige uma **equipe grande e dedicada**
- Empresas novas não têm áreas de funcionalidade suficientes para dividir em microsserviços

Empresa Pequena



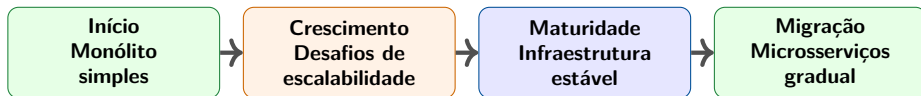
Monólito

comece simples!

Quando Migrar para Microsserviços?

Momento certo

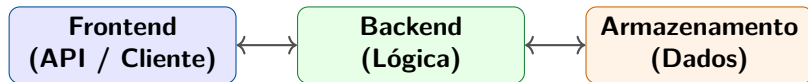
A migração se justifica quando os **desafios de escalabilidade** superam o custo de manter o ecossistema de microsserviços.



Não force microsserviços prematuramente!

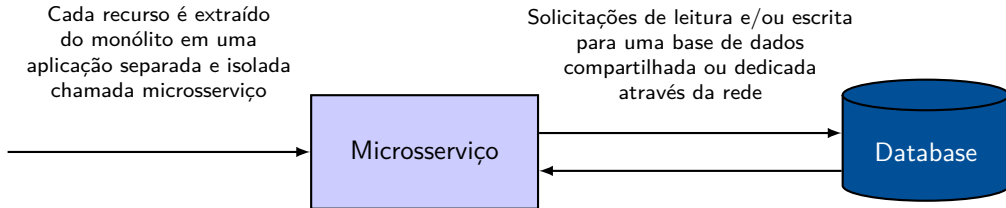
Componentes da Arquitetura

Todo microserviço terá **três componentes** principais:



- Esta estrutura não é muito diferente da arquitetura padrão de uma aplicação monolítica, mas aplicada em **microescala**.

Arquitetura Detalhada de um Microserviço



O Elemento Frontend (API)

O frontend de um microsserviço não é uma interface de usuário, mas uma **API (Interface de Programação de Aplicação)** com **endpoints estáticos**.

Exemplo de Endpoints (Serviço de Cliente)

- `get_customer_information`
- `update_customer_information`
- `delete_customer_information`

Na prática, todos esses endpoints convivem lado a lado como parte do **código de backend** que processa as solicitações.

A maioria dos microserviços armazena dados de duas formas principais:

Na Memória (Cache)

- Retorno imediato
- Sem chamadas de rede extras
- Alta velocidade

Em Database Externo

- Requer solicitação de rede externa
- Aguarda resposta do banco
- Maior persistência

Para que um conjunto de microsserviços funcione bem, certos elementos da arquitetura precisam ser **padronizados em toda a organização**.

- O **tipo de endpoint** deve ser o mesmo (ex: **REST** ou **Apache Thrift**).
- Misturar tipos (ex: REST e Thrift no mesmo serviço) é **raro** e torna o **monitoramento complicado**.
- A escolha dita a arquitetura. Ex: É difícil construir um serviço assíncrono comunicando-se via HTTP usando apenas REST puro.

Comunicação: Remote Procedure Calls (RPC)

Os microsserviços interagem entre si via **RPCs**.

O que são RPCs?

Chamadas através da rede projetadas para se comportarem **exatamente como chamadas locais de procedimentos**.

- Microsserviços com endpoints **REST** geralmente interagem via **HTTP**.
- Microsserviços com **Thrift** podem usar HTTP ou soluções internas personalizadas.
- O protocolo depende de **escolhas arquiteturais corporativas**.

Evite versionar microsserviços e endpoints!

Um microsserviço **não é uma biblioteca estática**, é uma **aplicação de software independente**.

Por que não versionar?

- Torna-se um **pesadelo organizacional** rapidamente.
- Clientes podem se prender a versões obsoletas e sem manutenção.
- Microsserviços devem ser tratados como **coisas vivas e em constante mutação**.

Decisões de Comunicação

Protocolos e endpoints são definidos pelo **projeto global** da empresa.

Lógica Interna

O desenvolvedor tem total liberdade para a lógica interna, incluindo a **escolha da linguagem** de programação (Java, Go, Erlang, Haskell...).

Ressalva: A liberdade extrema de linguagens tem um **custo elevado** para a organização de engenharia (manutenção, contratação, etc).

O Paradigma da Caixa-Preta

Devido à padronização das APIs e à liberdade interna, os microsserviços são tratados como **caixas-pretas**.



"Se você insere uma informação, obtém o que deseja em tempo razoável e sem erros, o serviço cumpriu o seu papel."

O Ecossistema de Microsserviços

Microsserviços **não vivem isolados**. O ambiente em que operam e interagem é o seu ecossistema.

Complexidade do Ambiente

As complexidades de um ambiente de microsserviços em grande escala são comparáveis às **complexidades ecológicas** de uma floresta tropical, um deserto ou um oceano.

- Considerar este ambiente como um **ecossistema** é altamente benéfico ao adotar a arquitetura.

Em ecossistemas sustentáveis e bem projetados, os microsserviços são **separados de toda a infraestrutura**. Eles são abstraídos de preocupações como:

- **Hardware** e servidores
- **Redes** e conectividade
- Pipeline de **build e deployment**
- **Descoberta de serviço** (Service Discovery)
- **Balanceamento de carga**

Tudo isso compõe a infraestrutura do ecossistema.

O Papel da Infraestrutura

A infraestrutura precisa **sustentar** o ecossistema de microsserviços de forma estável, escalável e tolerante a falhas.

Objetivo da Engenharia de Infraestrutura

Remover as questões operacionais de **baixo nível** das mãos dos desenvolvedores de microsserviços.

- Desenvolver um microsserviço deve parecer com desenvolver uma **aplicação autônoma pequena**.
- A infraestrutura subjacente deve ser muito **sofisticada** e de **alta qualidade**.

As Quatro Camadas do Ecosistema

O ecossistema pode ser dividido em quatro camadas (embora as fronteiras nem sempre sejam perfeitamente claras).

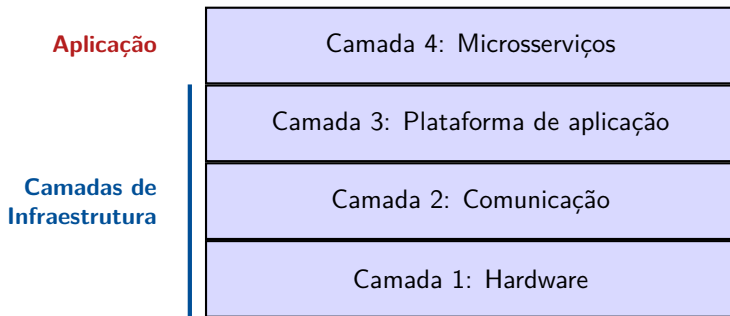


Figura 1.7. Modelo de quatro camadas do ecossistema de microserviços.

A Camada de Hardware

Na base do ecossistema de microsserviços temos a **camada de hardware**.

O que é?

Trata-se das máquinas propriamente ditas, os **computadores físicos reais** nos quais as ferramentas e os microsserviços são executados.

Características:

- Localizados em racks dentro de **datacenters**.
- Refrigerados por sistemas HVAC e alimentados por eletricidade.
- Podem ter diferentes especializações (ex: otimizados para banco de dados ou para processamento de CPU).

Os servidores podem ser físicos (próprios) ou alugados (nuvem).

Datacenters Próprios

- Controle total
- Escolha otimizada de hardware
- Alto custo inicial e manutenção

Provedores de Nuvem

- Cenário mais comum
- AWS (EC2), GCP, Azure
- Hardware limitado ao oferecido

A decisão envolve: Custo, Disponibilidade, Confiabilidade e Despesas Operacionais.

Gerenciar os servidores é papel fundamental desta camada.

- Cada servidor precisa de um **Sistema Operacional (OS) padronizado**.
- **A escolha do OS** depende das aplicações, linguagens e ferramentas.
- **Cenário comum:** Variantes de Linux (CentOS, Debian, Ubuntu).
- **Exceções:** Ambientes corporativos específicos (ex: .NET tradicional).

Provisionamento e Configuração

Após a instalação do OS, ferramentas de **gerenciamento de configuração** (Ansible, Chef, Puppet) garantem a padronização e automação na instalação de pacotes.

Sobre o hardware e o OS, outras abstrações são construídas:

- **Isolamento e abstração de recursos:** Tecnologias como Docker e Apache Mesos pertencem a esta camada.
- **Databases:** Bancos de dados dedicados ou compartilhados também operam aqui.

Monitoramento e Logging (em nível de servidor)

- Uso de ferramentas como **Nagios**.
- Essencial para diagnosticar, mitigar e resolver: falhas de disco, picos de CPU, quedas de rede.

Em resumo, a Camada 1 do ecossistema de microsserviços contém:

- ① Servidores **físicos** (próprios ou na nuvem)
- ② **Databases** (dedicados e/ou compartilhados)
- ③ **Sistema operacional** padronizado
- ④ Ferramentas de **isolamento e abstração de recursos**
- ⑤ **Gerenciamento de configuração**
- ⑥ **Monitoramento** em nível de servidor
- ⑦ **Logging** em nível de servidor

A Camada de Comunicação

A **camada de comunicação** trata de toda a interação entre os microsserviços.

Fronteiras Difusas

Esta camada se espalha por todas as outras camadas do ecossistema. As fronteiras entre a comunicação e a plataforma ou os serviços são pouco definidas.

Elementos contidos nesta camada:

- Rede e DNS (infraestrutura de base)
- RPCs (Remote Procedure Calls) e Endpoints de API
- Descoberta de Serviço (Service Discovery) e Registro
- Balanceamento de Carga

Como os Microsserviços Interagem?

A interação ocorre através da rede enviando dados em um **formato padronizado** (ex: JSON, Protocol Buffers). Existem dois paradigmas principais de comunicação:

1. HTTP + REST / Thrift

Comunicação direta baseada em solicitações e respostas síncronas.

2. Troca de Mensagens (Messaging)

Comunicação indireta assíncrona usando um *Broker* (intermediário).

Paradigma 1: HTTP + REST / Thrift

É o modelo **mais comum** e mais conveniente.

- Os serviços se comunicam via rede usando **HTTP**.
- Enviam solicitações (GET, POST, etc.) a endpoints REST ou Thrift.
- Dados formatados geralmente como JSON.

Vantagens e Desvantagens

- + Fácil de configurar, estável, previsível e difícil de implementar incorretamente.
- Por necessidade, é **síncrono (bloqueante)**. O serviço que chama fica aguardando a resposta.

Exemplo Prático: Endpoint REST (Express/Node.js)

O código abaixo ilustra a criação de um endpoint **REST síncrono** usando a biblioteca Express em JavaScript.

```
const express = require('express');
const app = express();
app.use(express.json());

// Endpoint de Consulta de Cliente
app.get('/api/customers/:id', (req, res) => {
  const customerId = req.params.id;
  // Logica para obter cliente do banco de dados (abstraida)
  const customer = { id: customerId, name: "Maria Silva", status: "Ativo" };

  if(customer) {
    res.status(200).json(customer); // Retorna dados em JSON
  } else {
    res.status(404).json({ error: "Cliente nao encontrado" });
  }
});

app.listen(3000, () => console.log('Servico Cliente na porta 3000'));
```

Exemplo Prático: Consumindo o Serviço

Outro microserviço precisa das informações do cliente. Ele fará uma **chamada síncrona** pela rede para o endpoint.

```
const axios = require('axios');

async function checkCustomerStatus(customerId) {
  try {
    // Faz a chamada HTTP GET para o Microserviço de Clientes
    const response = await axios.get(
      'http://servico-cliente:3000/api/customers/${customerId}'
    );

    console.log("Status do Cliente:", response.data.status);
    // Continua o processamento com a resposta recebida...

  } catch (error) {
    console.error("Falha ao se comunicar com o serviço de clientes");
  }
}

checkCustomerStatus(12345);
```

Paradigma 2: Troca de Mensagens (Messaging)

É um modelo **assíncrono (não bloqueante)**, porém mais complicado. **Como funciona?** Um microserviço envia dados para um **Broker de Mensagem**, que roteará a comunicação para outros microserviços.

- **Publish-Subscribe (Pub/Sub):** Assinatura de tópicos. Editores publicam, assinantes recebem.
- **Solicitação-Resposta:** Cliente envia pedido ao broker, que devolve a resposta.

Exemplos de Tecnologias: Apache Kafka, RabbitMQ, Redis.

Desafios da Troca de Mensagens

Embora seja altamente escalável (se bem projetada desde o início), apresenta desafios significativos:

- **Complexidade:** Não é tão fácil alterar e atualizar.
- **Ponto Único de Falha:** A natureza centralizada do broker pode comprometer todo o ecossistema se ele falhar.
- **Problemas de Concorrência:** A natureza assíncrona pode gerar *condições de corrida* e *loops infinitos* se a lógica não for muito bem elaborada.

Mitigação

Se implementado com proteções contra esses problemas, torna-se tão estável e eficiente quanto soluções síncronas.

Obrigado!

Dúvidas?